COMPILERS · QUIZ

Quiz Solution

Regular expressions · NFA → DFA · Derivations — Dr. Sherin Omran

حل الكويز بالتفصيل

بخطوات المحاضرات بس (Lecture 1, 2, 3-4)

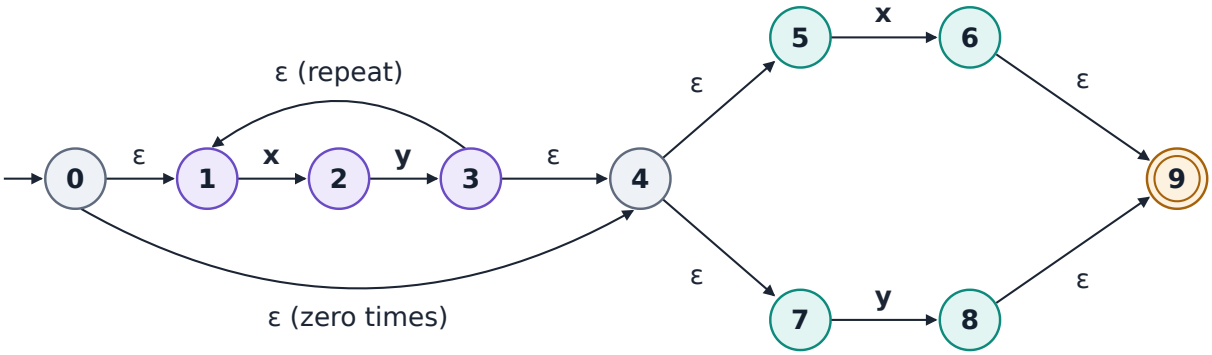
Q1
(xy)*(x|y) → NFA → DFA

Q2
aaabbbb · leftmost & rightmost

Q3
Two regular expressions

Q1 **Convert the following regular expression into an NFA, then convert from NFA to DFA: (xy)*(x|y)**

Step 1 — NFA using epsilon transition (Thompson construction)



Start state = 0 · Accepting state = 9 · purple: (xy)* · teal: (x|y)

Construction steps

1. **x , y:** each symbol is an NFA with two states and one edge labeled by the symbol.
2. **xy (concatenation):** the final state of x is the start state of y: 1 –x→ 2 –y→ 3.
3. **(xy)* (closure):** 0 –ε→ 1, ε from 3 back to 1 (repeat), 3 –ε→ 4, and ε from 0 to 4 (zero times).
4. **x|y (union):** state 4 has ε to 5 and to 7; 5 –x→ 6 , 7 –y→ 8; both go by ε to 9.
5. **(xy)*(x|y) (concatenation):** the end of the closure (state 4) is the start of the union — like state 7 in (a|b)*abb.

الفكرة الـ (xy)* بتتبني بنفس شكل الـ star في المحاضرة (ε راجعة + ε بتعدّي)، والـ union بيبدأ من نفس الـ state اللي الـ star خلصت فيها (state 4).

Lecture 3, 4 · Thompson's construction جه في

Q1 Step 2 — Convert from NFA to DFA (subset construction)

1. Begin with the start state 0 and calculate ε-closure(0)

A = ε-closure(0) = {0, 1, 4, 5, 7}

2. Find the states that A connects to

move(A, x) = {2, 6} → ε-closure = {2, 6, 9} = **B**

move(A, y) = {8} → ε-closure = {8, 9} = **C**

3. Examine state B

move(B, x) = ⌀ (no edge on x)

move(B, y) = {3} → ε-closure = {1, 3, 4, 5, 7} = **D**

4. Examine state C

move(C, x) = ⌀ , move(C, y) = ⌀

5. Examine state D

move(D, x) = {2, 6} → we have seen this before, it's **B**

move(D, y) = {8} → it's **C**

6. Accepting states

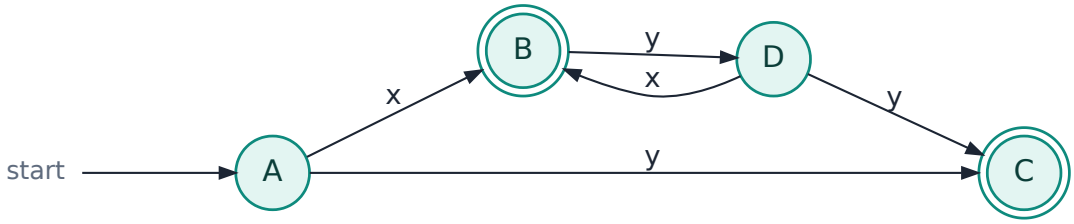
B = {2, 6, 9} and C = {8, 9} contain the NFA accepting state 9, so **B and C are accepting states**.

Transition table

State	Input x	Input y
→ **A**	B	C
B *	—	D
C *	—	—
D	B	C

→ start state · * accepting state · — no transition

Q1 The resulting DFA



There is only one edge from each state for a given input character. It's a DFA.

Check with some strings

String	Path	Result
x	A → B	✓ **accepted**
y	A → C	✓ **accepted**
xyx	A → B → D → B	✓ **accepted**
xyy	A → B → D → C	✓ **accepted**
xy	A → B → D	✗ **D not accepting**

الفكرة أي string لازم تخلص بحرف واحد x أو y بعد أي عدد من الـ xy، علشان كده الـ xy لوحدها مرفوضة.

Lecture 3, 4 · Subset construction **جه في**

Q2 **From the following grammar prove the string aaabbbb using leftmost and rightmost derivations: S → aSb | X , X → aX | Xb | a | b**

The string **aaabbbb** has 3 a's and 4 b's: S → aSb gives the outer **aa…bb**, and X gives the middle **abb**.

Leftmost derivation (replace the leftmost nonterminal each step)

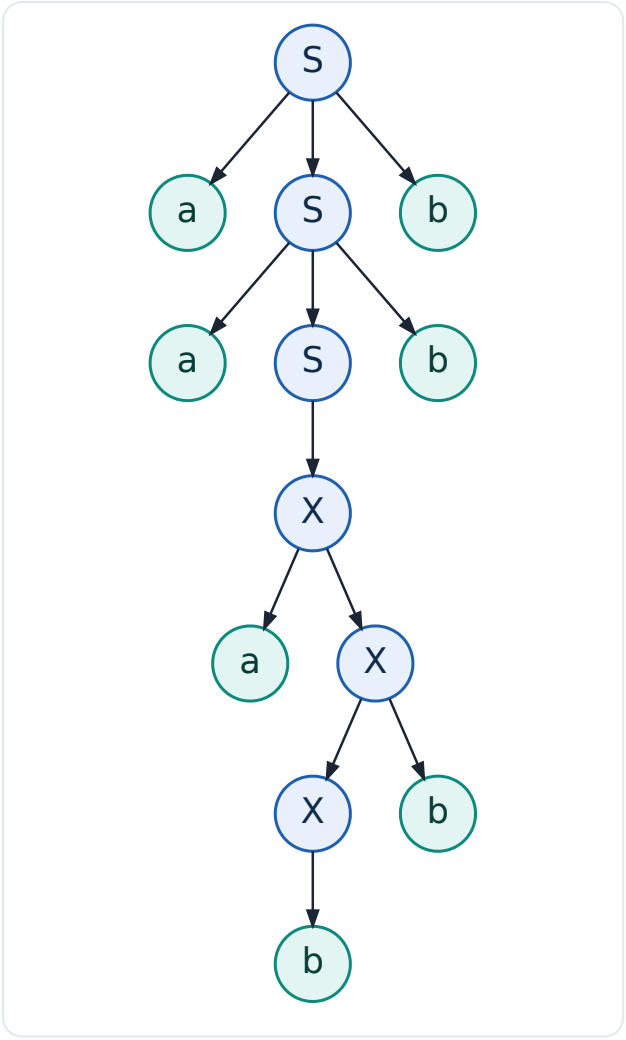
#	Derivation	Production used
1	S ⇒ aSb	S → aSb
2	⇒ aaSbb	S → aSb
3	⇒ aaXbb	S → X
4	⇒ aaaXbb	X → aX
5	⇒ aaaXbbb	X → Xb
6	⇒ aaabbbb	X → b

Rightmost derivation (replace the rightmost nonterminal each step)

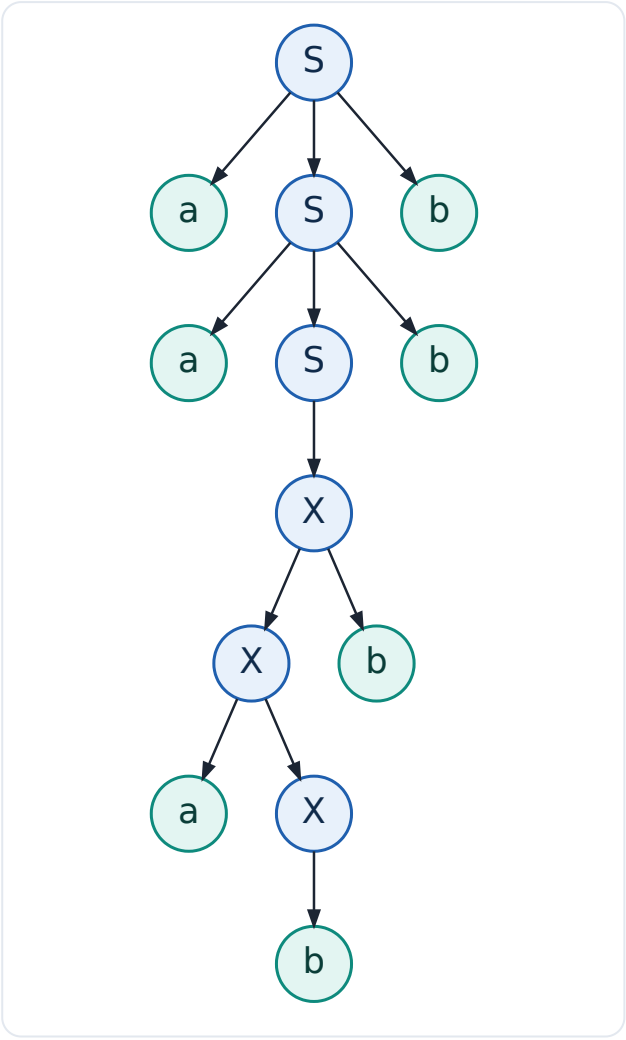
#	Derivation	Production used
1	S ⇒ aSb	S → aSb
2	⇒ aaSbb	S → aSb
3	⇒ aaXbb	S → X
4	⇒ aaXbbb	X → Xb
5	⇒ aaaXbbb	X → aX
6	⇒ aaabbbb	X → b

الفكرة في الـ leftmost بنستخدم X → aX الأول (الـ a على الشمال)، وفي الـ rightmost بنستخدم X → Xb الأول (الـ b على اليمين). الاتنين بيوصلوا لنفس الـ string.

Q2 Derivation (parse) trees for aaabbbb



Leftmost derivation tree



Rightmost derivation tree

1. The root is labeled by the start symbol **S**.
2. Each leaf is labeled by a terminal (a or b).
3. Each interior node is labeled by a nonterminal (S or X).
4. Reading the leaves from left to right gives **a a a b b b b** — the string is proved.

الفكرة الـ nonterminals باللون الأزرق والـ terminals باللون الأخضر. اقرأ الـ leaves من الشمال لليمين هتلاقيها aaabbbb.

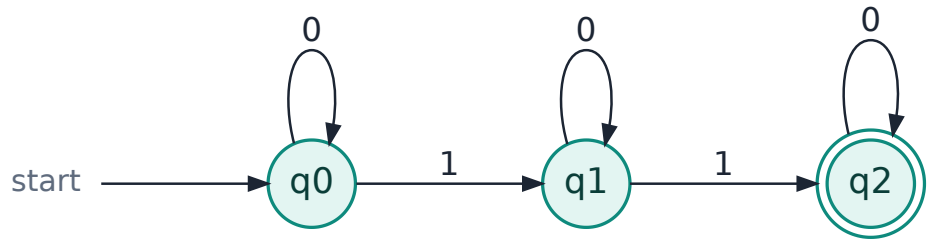
Q3 a **Write the regular expression that describes all strings having exactly two ones over the alphabet {0, 1}.**

R = 0* 1 0* 1 0*

1. **1** is written exactly two times, so every string has exactly two ones.
2. **0*** means zero or more 0's — before, between and after the two ones.

Accepted: 11, 0101, 1001, 00110 · **Rejected:** 1, 111, 0100

Transition diagram



Q3 b **Write a regular expression that represents an identifier in a programming language that can't start with a digit, in the alphabet containing digits and letters.**

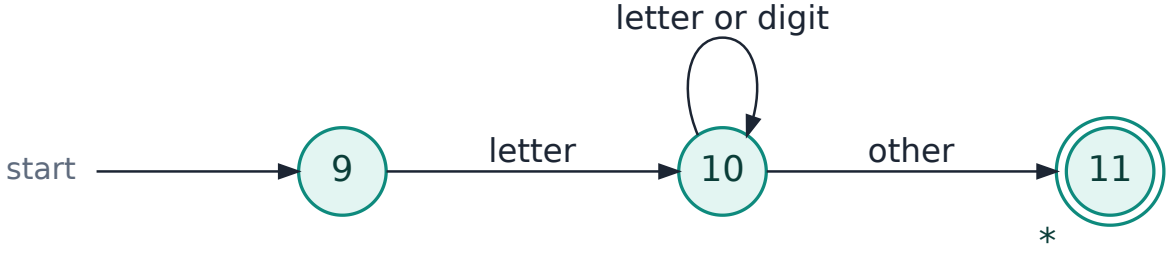
Let L be the set of letters {A, B, …, Z, a, b, …, z} and D be the set of digits {0, 1, …, 9}.

id = L (L | D)* i.e. letter (letter | digit)*

1. The first symbol must be a **letter**, so the identifier can't start with a digit.
2. **(letter | digit)*** means zero or more letters or digits after it.

Accepted: x, sum, a1, count2 · **Rejected:** 1a, 9x

Transition diagram for identifiers



Lecture 3, 4 · Transition diagram for identifiers Lecture 2 · L(L ∪ D)* جه في